

Project Development Report

AI-POWERED DOCUMENT ANALYZER

Application Overview & Tech Stack

The AI Document Analyzer is an intelligent, full-stack web application designed to extract, analyze, and interact with various document formats including PDFs, Word Documents, Excel sheets, text files, and images. It leverages advanced Large Language Models (LLMs) to automatically generate structured summaries, extract key entities, classify document types, and provide a conversational interface for users to query the contents of their uploaded files.

Technology Stack

React.js

Vite

Node.js

Express.js

Groq API (Llama 3.3/3.2)

Multer

pdf-parse & mammoth

Prompting Strategy & Frameworks

The application leverages the **Groq SDK** to interface with state-of-the-art Llama 3 models (`llama-3.3-70b-versatile` for text and `llama-3.2-90b-vision-preview` for images). The strategy relies heavily on **Zero-Shot Prompting** with strict constraints. By utilizing system prompts with distinct formatting directives, the application ensures the LLM's unstructured text is deterministically parsed for the React UI.

Sample Prompt: Document Analysis (JSON Enforced)

To populate the dashboard securely without parsing errors, the API call enforces `response_format: { type: 'json_object' }` alongside this prompt:

```
You are an expert document analyzer. Analyze the following extracted text from a document
Provide your analysis strictly in JSON format. The response must be a single JSON object.
Use the following keys:
- documentType: A short classification (e.g. "Invoice", "Contract", "Unknown").
```

- summary: A concise, beautifully structured markdown summary. Always use clear section titles.
- entities: An array of key-value pairs representing critical identifiers.
- confidence: A percentage integer (0-100) representing extraction confidence.

Extracted Document Text:

```
${documentText.slice(0, 15000)}
```

Sample Prompt: Conversational Chat Context

You are an AI assistant helping a user analyze a document named "\${docData.filename}". Use the following extracted document text as your primary context to answer the user's question. If the answer cannot be found in the text, politely state that the document does not contain the information.

FORMATTING REQUIREMENTS:

- Always format your answers using clean Markdown with distinct section headings.
- EVERY bullet point MUST be placed on its OWN NEW LINE starting with "- ".

Phase-by-Phase Development Summary

Phase 1: Project Setup & Architecture

Initialized a monorepo-style structure separating `frontend` and `backend` environments. Configured Vite for the React client and set up the Express server with essential middleware like CORS and body parsers.

Phase 2: Core Backend Engine & Extraction

Implemented file upload handling via `multer`. Integrated various parsing libraries (`pdf-parse`, `mammoth` for .docx, `xlsx` for spreadsheets) to extract raw text. Hooked up the Groq API to process this text and implemented the Llama 3 Vision model as a fallback for OCR on image files.

Phase 3: Frontend Development & UI

Built a responsive React UI utilizing modern aesthetic principles, including dark mode, glassmorphism, and interactive backgrounds. Developed the Upload Dashboard and a split-view Analysis Page that features a document preview pane alongside the AI chat interface.

Phase 4: Refinement & Cloud Deployment

Addressed complex deployment issues on Render. Fixed build failures caused by relative local workspace dependencies (`file:..`) and ensured environment variables (`VITE_API_URL` , `GROQ_API_KEY`) were properly mapped for production environments.

Application Architecture

The project follows a decoupled Client-Server architecture:

- **Client (Frontend):** A React Single Page Application (SPA). It manages application state, routing, and user interactions, making RESTful HTTP requests to the backend via `fetch`.
- **Server (Backend):** An Express.js Node app acting as a broker between the client and AI services.
 - `POST /api/upload`: Receives multipart form data, delegates to the appropriate file parser based on mime-type, requests AI summarization, and stores metadata.
 - `POST /api/chat`: Receives user queries and historical context, requesting conversational responses from the Groq API.
- **Data Persistence:** A lightweight local `data/` directory mechanism using a `history.json` file to maintain session state (designed for ephemeral cloud deployments).

Challenges & Resolutions

- **Challenge:** Extracting text from image-based or non-standard documents.
Resolution: Integrated the `llama-3.2-90b-vision-preview` model, effectively adding robust AI-powered Optical Character Recognition (OCR) directly into the upload pipeline.
- **Challenge:** Ensuring the LLM returns consistently parsable JSON for the frontend dashboard to render correctly.
Resolution: Leveraged Groq's JSON mode (`response_format: { type: 'json_object' }`) combined with an explicit JSON schema definition directly within the system prompt.
- **Challenge:** Deployment failing on PaaS (Render) due to `npm install` errors.
Resolution: Identified that the `"pdf-analyzer-workspace": "file:.."` package link works locally but breaks cloud build steps. Stripped these invalid dependencies and updated the lockfiles to resolve pipeline failures.

Key Learnings & Reflection

Building this application reinforced several critical concepts in modern full-stack AI development:

- **Prompt Engineering is Code:** Structuring prompts with strict rules and constraints is just as vital as writing the actual backend logic. The bridge between unstructured LLM text generation and deterministic UI rendering relies entirely on robust prompt design.
- **File Handling Nuances:** Managing diverse file types (PDF vs. Word vs. Excel) requires a modular approach. Relying on specialized parsers ensures the LLM receives clean text, which drastically improves the quality of the analysis.
- **Cloud Environment Parity:** The discrepancies between local development (where relative folder structures exist) and containerized cloud deployments emphasize the importance of container-agnostic configurations and avoiding local package linking for production builds.